

qliff

A Clifford and noisy stabilizer simulator

Stim-class clean speed, and the coherent noise stim cannot touch.

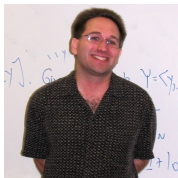
What is qliff?

- ▶ A **stabilizer simulator**: simulates a class of quantum circuits without approximation.
- ▶ It tracks $\sim n^2$ bookkeeping, not 2^n amplitudes.
- ▶ Two jobs:
 - ▶ **clean Clifford** at stim speed, and
 - ▶ **noisy** simulation including coherent noise stim cannot do.
- ▶ Ships decoder-ready **QEC** outputs (detectors, observables, error model).

narrow **Rust** core: the tableau hot path

rich **Python**: everything you extend

Built on work from



Scott Aaronson

UT Austin

*Quantum complexity,
CHP, BosonSampling*



Daniel Gottesman

Univ. of Maryland
(QuICS)

*Stabilizer formalism, fault
tolerance*



Hui Khoon Ng

NUS (CQT), Singapore

Approximate QEC



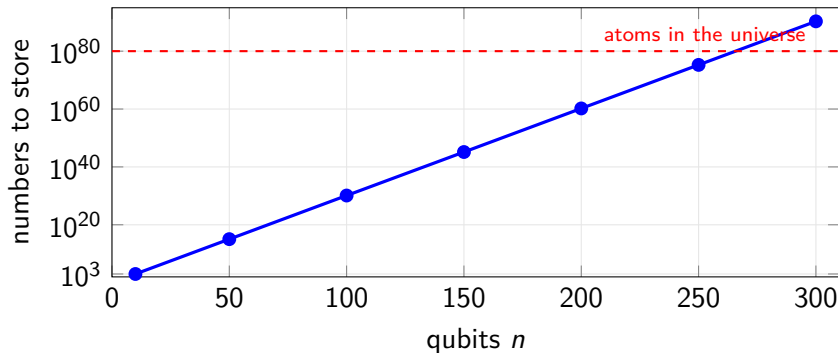
Craig Gidney

Google Quantum AI

Stim, Cirq, Sparse Bloom

Why is simulating qubits hard?

A qubit is a blend of 0 and 1, so n qubits need 2^n numbers to write down in full.



Past ~ 270 qubits the state has more numbers than the universe has atoms. You need a shortcut.

What is a stabilizer?

Plain idea: name a state by the operations that leave it unchanged.

state	stabilized by ($S \psi\rangle = \psi\rangle$)
$ 0\rangle$	$+Z$
$ 1\rangle$	$-Z$
$ +\rangle = \frac{1}{\sqrt{2}}(0\rangle + 1\rangle)$	$+X$
Bell $\frac{1}{\sqrt{2}}(00\rangle + 11\rangle)$	$+XX$ and $+ZZ$

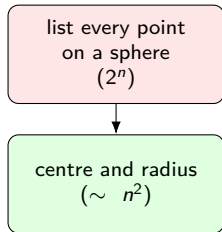
An n -qubit state is pinned down by n such operators, its *generators*.

The shortcut: track the rules, not the state

- ▶ For **Clifford** circuits you can drop amplitudes.
- ▶ Keep only the n stabilizers of the state.
- ▶ Cost falls from 2^n to $\sim n^2$ (*Gottesman–Knill*).

The catch. No room for:

- ▶ non-Clifford gates (T , Toffoli)
- ▶ arbitrary superpositions
- ▶ mixed states



The alphabet (this is the whole trick)

symbol	meaning
I	do nothing
X	flip the bit
Z	flip the sign
Y	both

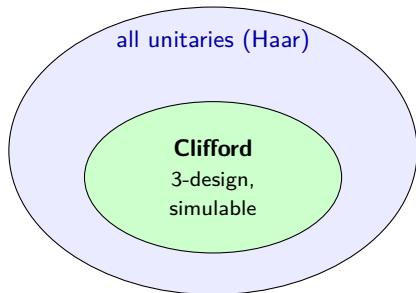
Each symbol is **2 bits** (x, z) with one sign bit.

gate = rewrite bits with XOR and AND

No floating point, so it packs 64 qubits per machine word.

Why stabilizer simulators are everywhere

Plain idea: random Clifford circuits mimic Haar-random ones, yet stay classically simulable.



A **unitary 3-design** matches Haar-random unitaries up to the 3rd statistical moment. The Clifford group is one. So it stands in for “random circuit” in:

- ▶ randomized benchmarking
- ▶ classical shadows, tomography
- ▶ error correction (codes are Clifford)

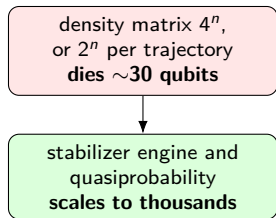
Two kinds of noise

	Pauli noise	Coherent / damping
what it is	random X , Z flips	systematic tilt, energy loss
stabilizer formalism	stays inside	breaks it
stim	✓	×
qliff	✓	✓ (quasiprobability)

qliff does the hard kind by a weighted-average method, nearly as cheap as the easy kind. **This is the differentiator.**

Why?

Plain idea: noisy simulation at thousands of qubits, which a state-vector simulator cannot reach. Required for the Edge effects work



Myers, Teo, Mishra, Chai, Ng

Simulating general noise nearly as cheaply as Pauli noise

arXiv:2512.07304 (NUS)

- ▶ any channel becomes signed stabilizer ops
- ▶ stratified importance sampling

The two jobs, concretely

Clean, Pauli (like stim)

- ▶ bit-packed tableau, frame sampling

```
Circuit().H(0).CX(0,1).M(0,1)
```

Coherent (stim cannot)

- ▶ $E = \sum_{\mu} q_{\mu} S_{\mu}$, signed weights
- ▶ importance-sample $\langle O \rangle$

```
c.RZ(0,  $\theta$ ).estimate("X")  $\approx \cos \theta$ 
```

One engine: the noisy estimator runs many pure-Clifford trajectories and averages them.

The tableau (CHP)

$2n + 1$ rows: n destabilizers, n stabilizers, 1 scratch row.

one row:

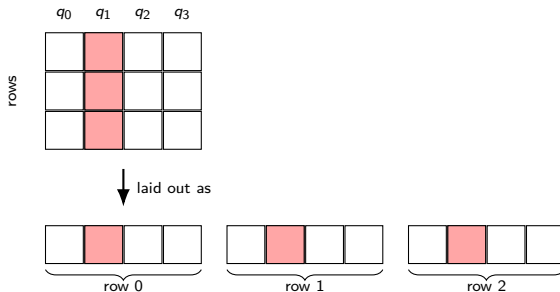
x bits $[n]$	z bits $[n]$	sign r
----------------	----------------	----------

(x_q, z_q) : 00= I , 10= X , 01= Z , 11= Y

x, z are $\text{Vec}\langle\text{u64}\rangle$, $W = \lceil n/64 \rceil$ words per row. The scratch row is bookkeeping: it holds the running measurement product so the answer falls out in place.

Row-major: store the grid one row at a time

The tableau is a grid: one **row** per (de)stabilizer, one **column** per qubit. Memory is 1D, so we pick an order.

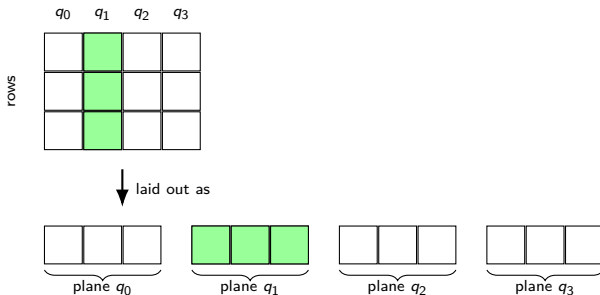


A **gate** hits one qubit across all rows (a column), so its cells are **strided**; a **measurement** combines whole rows, which sit **contiguous**.

So row-major is the fast layout for measurement. Gates fall off the **cache cliff** at large n (Wall 1).

Column-major: store the grid one qubit-plane at a time

Same grid, swapped order: write each qubit's whole column before moving to the next.



Now a **gate** sweeps one qubit-plane, all **contiguous** (cache cliff gone); a **measurement** must hop plane to plane, so it is **strided**.

So column-major is the fast layout for gates. We keep both layouts and transpose at the gate/measurement boundary.

The map: two loops, two walls

Inner loop: the tableau

cost of one Clifford gate or measurement

- ▶ faster measurement (combine rows wordwise)
- ▶ one-pass CZ
- ▶ **Wall 1**: gates at large n , fixed by column-major
- ▶ batch across the Python/Rust border

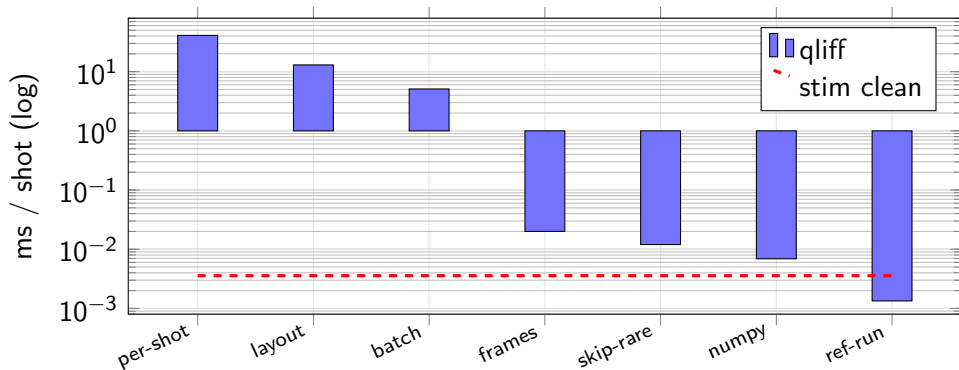
Outer loop: noisy sampling

many shots of one noisy circuit

- ▶ **Wall 2**: re-running per shot, fixed by Pauli frames
- ▶ skip the shots that do not fault
- ▶ zero-copy numpy output
- ▶ run on all CPU cores

Then the payoff: the coherent-noise estimator, the thing stim cannot do, moved into parallel Rust.

Noisy sampling: the wins stack up ($n = 1024$)



Each bar applies one more technique. From 41 to 0.0013 ms/shot, about **30000** $\times$, dropping below stim's clean sampler.

Opt 1: combine rows 64 bits at a time

Was slow: a measurement combines two tableau rows qubit-by-qubit, a serial $O(n)$ loop.

per-qubit loop, $O(n)$ $\implies$ whole words at once (bitwise ops and a popcount),
 $O(n/64)$

Measurement flips from $2.6\times$ **slower** than stim to $2\times$ **faster**.

Opt 2: one-pass CZ

Was slow: CZ was built from H, CX, H, three full sweeps over a qubit-plane.

H, CX, H (3 sweeps) $\implies$ one symmetric update (1 sweep)

2 $\times$ **faster CZ**; surface codes are CZ-heavy. Verified over 400 circuits.

Opt 3: Wall 1, the cache cliff (column-major)

Was slow: in row-major a gate strides across rows; past $n \approx 512$ every read misses cache.

gate path at $n = 1024$	vs stim
row-major (strided reads)	$\sim 11\times$ slower
column-major (contiguous planes)	$1.2\times$ faster

Contiguous planes erase the stride: raw tableau ~ 10.7 M/s, about $4\times$ stim at $n = 1024$.

Opt 4: batch across the Python/Rust border

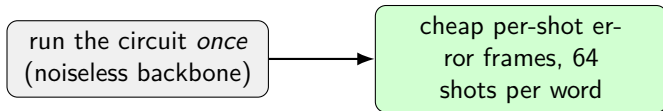
Was slow: each gate was its own call from Python into Rust; at millions of gates/s that border dominates.

one call per gate $\implies$ one call per block of gates

- ▶ `Circuit.run` batches to ~ 20 M/s at $n = 1024$, about $8\times$ stim.
- ▶ the whole noisy Monte-Carlo loop now runs inside Rust.
- ▶ custom Python noise channels still work, on a slower fallback path.

Opt 5: Wall 2, the Pauli-frame sampler

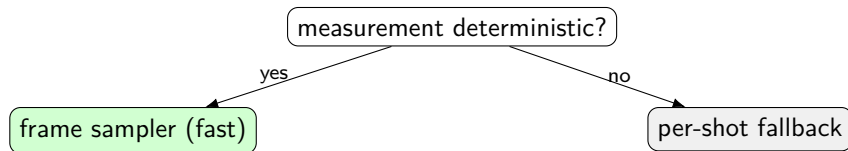
Was slow: noisy sampling re-ran the entire circuit from scratch, once per shot.



41 ms/shot $\implies$ **0.020 ms/shot**, about $5\times$ stim clean.

Opt 5 (cont.): when frames do not apply

The catch: frames assume the noiseless run fixes each measurement, with noise only flipping it.



A random measurement cannot ride along in a frame, so it routes to the per-shot path. QEC syndromes are deterministic, so they stay fast.

Opt 6: skip the shots that do not fault

Was slow: we flipped a coin for every (noise location, shot) pair, even though faults are rare.

draw the gap to the next faulting shot, then jump straight there

touch only $\sim \phi \cdot \text{shots}$, a $\sim 1/\phi$ win. At $\phi = 10^{-3}$: $0.020 \rightarrow \mathbf{0.012 \text{ ms/shot}}$.

Opt 7: hand back raw bytes, not Python ints

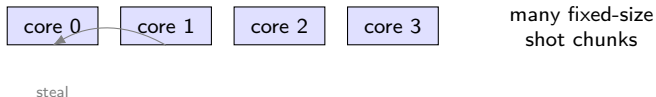
Was slow: each measured bit was boxed into a Python integer, millions of allocations per sample.

~5M boxed Python ints $\implies$ one flat byte buffer viewed as a numpy array
(zero copy)

Killed ~40% of per-shot time: 0.012 $\rightarrow$ **0.0069 ms/shot.**

Opt 8: use every CPU core

Was slow: the sampler ran on a single core while the rest sat idle.



Idle cores steal chunks from busy ones. **2.6×** on 8 cores at 200k shots; fixed chunks keep results reproducible.

Opt 9: the hard-noise estimator, in parallel Rust

The payoff: coherent and general noise (which stim cannot do) ran as a slow Python loop.

coherent / general $\langle O \rangle$: Python loop $\implies$ multicore Rust

229 $\times$ faster (5793 ms $\rightarrow$ 25 ms), validated unbiased.

$R_Z(\theta) \rightarrow \langle X \rangle = \cos \theta$; amplitude damping on $|1\rangle \rightarrow \langle Z \rangle = 2p - 1$.

1 qubit: the X gate

State $|0\rangle$, stabilizer $+Z$. Rule: $r \oplus = z$.

	x	z	r	Pauli
before	0	1	0	$+Z$, stabilizes $ 0\rangle$
after	0	1	1	$-Z$, stabilizes $ 1\rangle$

$z = 1$, so the sign flips: $X|0\rangle = |1\rangle$. X flips any row with a Z -component.

1 qubit: the H gate

State $|0\rangle$, stabilizer $+Z$. Rule: swap $x \leftrightarrow z$, then $r \oplus = xz$.

	x	z	r	Pauli
before	0	1	0	$+Z$, stabilizes $ 0\rangle$
after	1	0	0	$+X$, stabilizes $ +\rangle$

$xz = 0$, so no sign flip; the swap turns Z into X : $H|0\rangle = |+\rangle$.

2 qubits: X on one qubit

State $|00\rangle$, generators $+ZI$ and $+IZ$. Apply X on qubit 0 ($r \oplus = z_{q0}$).

generator	$x(q_0q_1)$	$z(q_0q_1)$	r	Pauli
g_1 before	00	10	0	$+ZI$
g_1 after	00	10	1	$-ZI$
g_2	00	01	0	$+IZ$, untouched

Only g_1 has $z_{q0} = 1$, so only its sign flips: $|00\rangle \rightarrow |10\rangle$.

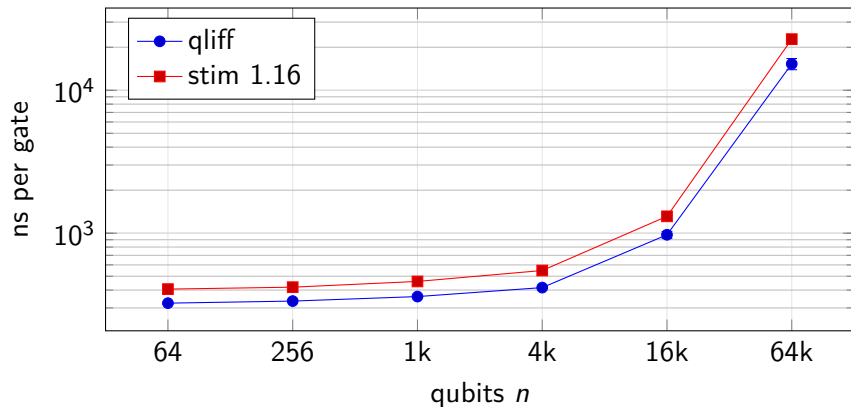
2 qubits: CX builds a Bell pair

$$|00\rangle \xrightarrow{H(0)} \xrightarrow{CX(0 \rightarrow 1)} \text{Bell state.}$$

after $H(0)$	apply $CX(0 \rightarrow 1)$	result
$+XI$	$x_1 \oplus = x_0: XI \rightarrow XX$	$+XX$
$+IZ$	$z_0 \oplus = z_1: IZ \rightarrow ZZ$	$+ZZ$

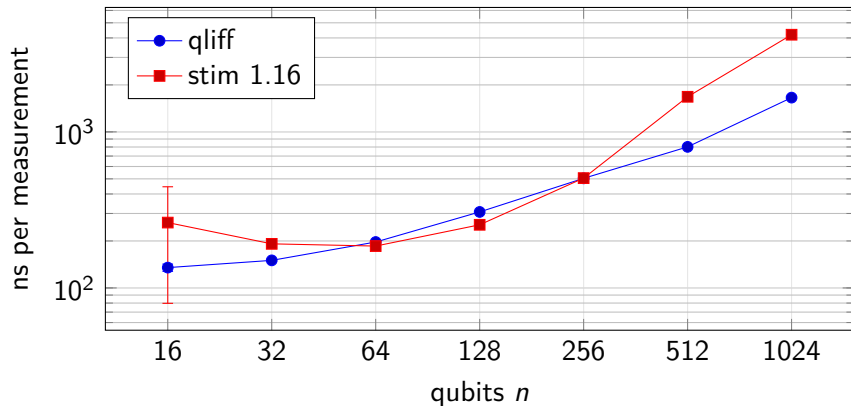
$\{+XX, +ZZ\}$ is the Bell state. Matches `Simulator(2).H(0).CX(0,1).canon() = ['+XX', '+ZZ']`.

Gate throughput (ns per gate), versus stim



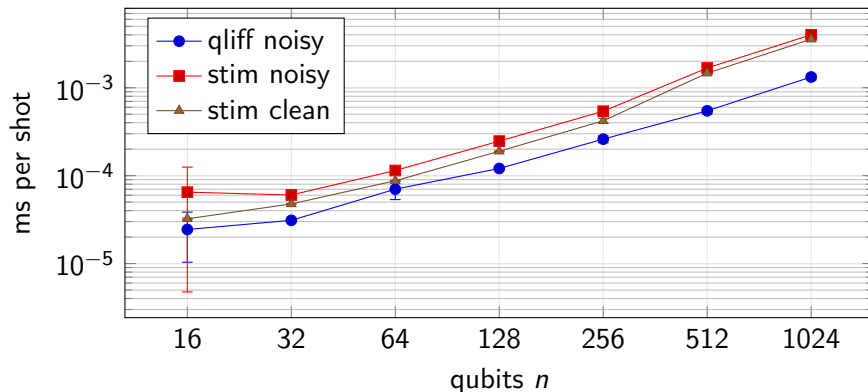
No crossover: qliff leads at every size and the gap widens, **1.5 $\times$** at $n = 65536$. (Single-gate dispatch, mix 34% H / 33% S / 33% CX.)

Measurement throughput (ns per measurement)



Crossover near $n = 64$ to 256; at scale qliff pulls ahead, $2.5\times$ at $n = 1024$.

Noisy sampling (ms per shot), versus stim clean



qliff noisy beats stim clean at every size, $2.7\times$ at $n = 1024$. (DEPOLARIZE1, $p = 10^{-3}$.)

Feature comparison

capability	qliff	stim
Clifford tableau, mid-circuit measure, reset, feedback	✓	✓
Bit-packed frame sampling, multicore	✓	✓
Pauli noise (depolarize, X/Z error, Pauli channel)	✓	✓
Detectors, observables, DEM export for decoders	✓	✓
Coherent and amplitude-damping noise	✓	×
Quasiprobability importance estimator	✓	×
Python-first custom channels, no recompile	✓	×
Circuit file format, diagrams, large gate zoo	×	✓
Tableau, PauliString, Flow algebra objects	×	✓
Many built-in code families, mature ecosystem	2 codes	✓